Bài 1.3 – NumPy: mảng và phép toán vector hóa

> **Giai đoạn 1 – Python cho ML & Data Science** · Thời lượng: 3 buổi · Hình thức mở đầu: *Thử thách có tính giờ*

Chia buổi:

- **Buổi 1:** Mục 1 → 6.6 (vì sao cần NumPy, tạo mảng, dtype, lấy phần tử).
- **Buổi 2:** Mục 6.7 → 6.14 (lọc, tính toán, thống kê theo trục, sắp xếp, đổi hình dạng, ghép mảng).
- **Buổi 3:** Mục 6.15 → hết (broadcasting, view/copy, ngẫu nhiên, nhân ma trận, lỗi thường gặp, trắc nghiệm, thực hành).

---

1. 🎯 Mục tiêu bài học

Sau bài này bạn có thể...

- Giải thích được vì sao mọi thư viện ML (Pandas, Scikit-learn, PyTorch, OpenCV) đều xây trên NumPy.
- Tạo mảng 1, 2, 3 chiều, đọc được `shape`, `dtype`, và chọn đúng kiểu số cho dữ liệu.
- Lấy, cắt, lọc dữ liệu bằng chỉ số, lát cắt, danh sách chỉ số và điều kiện (boolean mask).
- Tính toán và thống kê trên cả mảng **không cần vòng lặp**, theo hàng hoặc theo cột (`axis`).
- Hiểu **broadcasting**, **view và copy**, hai nguồn gây lỗi "âm thầm" nhiều nhất, và tự sửa được các lỗi `shape` hay gặp.

---

2. 🔗 Ôn nhanh kiến thức cần có

- **List** (bài 1.1): `ds = [2.5, 3.1, 4.0]`, lấy phần tử `ds[0]`, cắt lát `ds[1:3]`, chỉ số âm `ds[-1]`.
- **List lồng nhau** (bài 1.1): `[[50, 2], [65, 2]]` là một "bảng" 2 hàng.
- **Vòng lặp và comprehension** (bài 1.1): `[x * 2 for x in ds]`.
- **Hàm và class** (bài 1.2): `np.array` là một class, `.mean()` là một method.
- **Hai biến cùng trỏ vào một list** (mini-game bài 1.1): sửa biến này thì biến kia cũng đổi. Ta sẽ gặp lại hiện tượng này ở mục 6.16.

---

3. 🧩 Mở đầu: Thử thách có tính giờ ⏱️

Bạn là kỹ sư dữ liệu ở một sàn bất động sản. Mỗi đêm hệ thống phải tính lại "điểm" cho **1 triệu** tin đăng. Bước đầu tiên chỉ là bình phương từng số rồi cộng lại. Liệu cách viết có quan trọng không?

Đề: Tính tổng bình phương của 1 triệu số: $0^2 + 1^2 + 2^2 + \ldots + 999.999^2$. Dán vào Colab và chạy:

```
import time                 # thư viện đo thời gian
import numpy as np          # NumPy, quy ước luôn đặt tên tắt là np

n = 1_000_000               # dấu _ chỉ để dễ đọc, Python bỏ qua

# --- Cách 1: vòng lặp Python ---
```

```
ds = list(range(n))
t0 = time.time()                        # bấm giờ
tong_vong_lap = 0
for x in ds:
    tong_vong_lap += x * x
t1 = time.time()

# --- Cách 2: NumPy ---
mang = np.arange(n, dtype=np.int64)     # mảng 0, 1, 2, ..., 999999
tong_numpy = (mang * mang).sum()        # bình phương CẢ MẢNG một lúc rồi cộng
t2 = time.time()

print("Vòng lặp:", tong_vong_lap, f"- {t1 - t0:.4f} giây")
print("NumPy   :", tong_numpy, f"- {t2 - t1:.4f} giây")
print(f"NumPy nhanh hơn khoảng {(t1 - t0) / (t2 - t1):.0f} lần")
```

Kết quả mẫu (thời gian trên máy bạn sẽ khác):

```
Vòng lặp: 333332833333500000 - 0.0770 giây
NumPy   : 333332833333500000 - 0.0056 giây
NumPy nhanh hơn khoảng 14 lần
```

Cùng một đáp số, nhưng NumPy nhanh hơn **hàng chục lần** (thường 10–100 lần tùy máy). Với dữ liệu ML thật, hàng triệu dòng, hàng nghìn cột, huấn luyện lặp lại hàng trăm lần, khác biệt này là **vài phút so với vài giờ**.

Câu hỏi đặt ra cho cả bài: **NumPy làm gì mà nhanh như vậy, và viết code theo kiểu NumPy là viết thế nào?**

---

4. 💡 Trực giác

4.1. List Python là "tủ hồ sơ", mảng NumPy là "khay trứng"

- **List Python** giống một tủ hồ sơ: mỗi ngăn chứa **một tờ giấy ghi địa chỉ**, chỉ tới món đồ thật nằm ở chỗ khác trong kho. Món đồ có thể là số, chữ, thậm chí một list khác. Muốn cộng 1 triệu số, Python phải chạy đi chạy lại 1 triệu lần: mở ngăn, đọc địa chỉ, ra kho lấy đồ, kiểm tra "đây là số gì?", rồi mới cộng.
- **Mảng NumPy** giống một khay trứng: các quả trứng **cùng loại, cùng kích thước, xếp sát nhau**. Người công nhân không cần kiểm tra từng quả. Họ đặt cả khay vào máy và máy xử lý cả khay một lượt.

```
List Python:  [ ●→int ][ ●→float ][ ●→int ] ...  mỗi ô là một "mũi tên" trỏ tới dữ liệu nằm rải rác
              Python phải kiểm tra kiểu từng phần tử, từng bước một.

Mảng NumPy:   [ 0 | 1 | 2 | 3 | 4 | ... ]       các số nằm sát nhau trong bộ nhớ, cùng một kiểu
              Phép tính chạy bằng code C đã tối ưu, xử lý cả khối một lúc.
```

4.2. Vector hóa: ra lệnh cho "cả đội", không phải từng người

Giáo viên muốn cộng 1 điểm cho cả lớp 40 học sinh.

- **Cách vòng lặp:** gọi từng em lên bảng, sửa điểm, cho về chỗ. 40 lần.
- **Cách vector hóa:** nói một câu "cả lớp cộng 1 điểm". Xong.

NumPy cho bạn viết `diem + 1` thay vì viết vòng lặp `for`. Code **ngắn hơn**, **dễ đọc hơn**, và **nhanh hơn**. Ba lợi ích cùng lúc.

4.3. Broadcasting: "photo" cho đủ số lượng

Bạn có bảng điểm 40 học sinh × 3 môn và muốn trừ điểm trung bình của từng môn. Bạn chỉ có **1 hàng** gồm 3 con số trung bình. NumPy tự "photo" hàng đó thành 40 bản, đặt chồng lên bảng điểm, rồi trừ từng ô. Bạn không phải tự sao chép. Đó là **broadcasting**.

---

5. 📖 Lý thuyết chi tiết

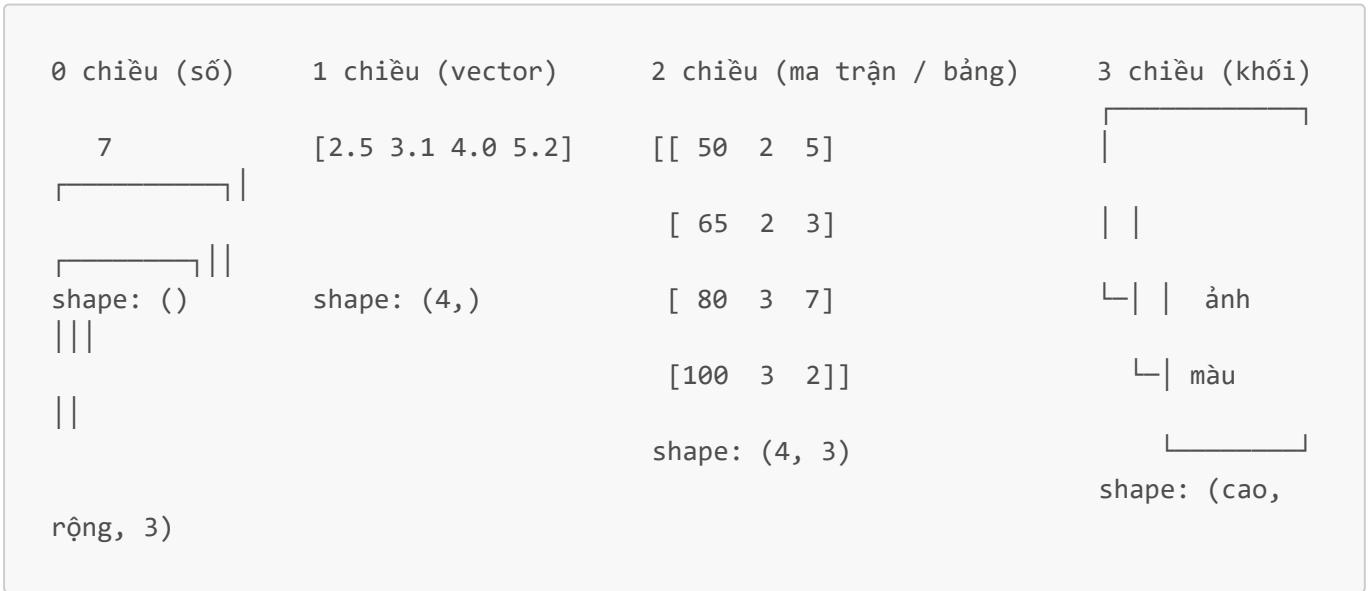
5.1. Mảng (ndarray) là gì?

ndarray – N-dimensional array (mảng nhiều chiều) là một khối số **cùng kiểu**, xếp theo dạng lưới đều đặn.

Một mảng luôn có 3 thông tin "căn cước":

Thuộc tính	Ý nghĩa	Ví dụ
`shape`	Kích thước theo từng chiều	`(4, 3)` = 4 hàng, 3 cột
`ndim`	Số chiều = độ dài của `shape`	`2`
`dtype`	Kiểu của **mọi** phần tử	`int64`, `float64`, `bool`

Hình dung theo số chiều:



Số chiều	Hình dung	Ví dụ trong ML
1	Một hàng số	Giá của 4 căn nhà; nhãn y
2	Bảng Excel	4 căn nhà × 3 đặc trưng (ma trận dữ liệu X)
3	Chồng nhiều bảng	Ảnh màu: cao × rộng × 3 kênh màu (RGB)
4	Nhiều khối	Một lô (batch) 32 ảnh: 32 × cao × rộng × 3 (sẽ gặp ở bài 3.10)

5.2. Trục (axis): đọc `shape` như đọc địa chỉ

Mỗi vị trí trong `shape` là một **trục (axis)**, đánh số từ 0.

```
X.shape = (4, 3)
           │  └── axis 1: đi NGANG qua các cột (3 cột)
           └───── axis 0: đi DỌC xuống các hàng (4 hàng)

              axis 1 ──────────►
           ┌  [  50    2    5 ]
  axis 0   │  [  65    2    3 ]
     │     │  [  80    3    7 ]
     ▼     └  [ 100    3    2 ]
```

Quy tắc vàng: khi bạn tính `X.sum(axis=0)`, trục 0 **biến mất**. Bảng `(4, 3)` còn lại `(3,)`, tức mỗi cột ra 1 số. Tính `axis=1` thì trục 1 biến mất, còn `(4,)`, tức mỗi hàng ra 1 số.

Tính tay với bảng nhỏ:

```
A = [[1, 2, 3],
     [4, 5, 6]]          shape (2, 3)

A.sum(axis=0) = [1+4, 2+5, 3+6] = [5, 7, 9]      shape (3,)   ← gộp 2 hàng
A.sum(axis=1) = [1+2+3, 4+5+6]  = [6, 15]        shape (2,)   ← gộp 3 cột
A.sum()       = 21                                             ← gộp tất cả
```

5.3. Kiểu dữ liệu (dtype)

Mỗi số trong mảng chiếm một số **bit** cố định trong bộ nhớ (1 byte = 8 bit).

dtype	Lưu được gì	Bộ nhớ / phần tử	Hay dùng cho
`int64`	Số nguyên rất lớn (±9,2 tỷ tỷ)	8 byte	Đếm, chỉ số, ID
`int32`	Số nguyên đến ±2,1 tỷ	4 byte	Tiết kiệm bộ nhớ
`uint8`	Số nguyên **0 đến 255**	1 byte	**Pixel ảnh** (CV)

dtype	Lưu được gì	Bộ nhớ / phần tử	Hay dùng cho
`float64`	Số thực, ~15–16 chữ số chính xác	8 byte	Mặc định của NumPy, tính toán thống kê
`float32`	Số thực, ~7 chữ số chính xác	4 byte	**Deep Learning** (PyTorch mặc định dùng float32)
`bool`	`True` / `False`	1 byte	Mặt nạ lọc dữ liệu
`<U10`	Chuỗi tối đa 10 ký tự	40 byte	Ít dùng; chữ nên để Pandas xử lý

Tính thử bộ nhớ: bảng 1 triệu dòng × 100 cột:

- `float64`: 1.000.000 × 100 × 8 byte = 800.000.000 byte ≈ **800 MB**.
- `float32`: 1.000.000 × 100 × 4 byte ≈ **400 MB**. Tiết kiệm một nửa, đủ chính xác cho Deep Learning.

5.4. Ba quy tắc broadcasting

Khi tính giữa hai mảng khác `shape`, NumPy so sánh `shape` **từ phải sang trái**, từng cặp kích thước:

1. Hai kích thước **bằng nhau** → OK.
2. Một trong hai bằng **1** → kích thước 1 được "kéo giãn" cho bằng bên kia.
3. Thiếu chiều thì coi như có thêm chiều kích thước **1** ở **bên trái**.

Nếu một cặp nào đó không thỏa → lỗi `ValueError: operands could not be broadcast together`.

Tính tay 3 ví dụ:

```
Ví dụ 1:  (4, 3)  với  (3,)
          (4, 3)
          (1, 3)   ← quy tắc 3: thêm 1 bên trái
           ↓  ↓
           4  3    ← cặp phải: 3 = 3 OK; cặp trái: 1 kéo giãn thành 4  →  kết quả
(4, 3) ✅

Ví dụ 2:  (4, 3)  với  (4,)
          (4, 3)
          (1, 4)   ← thêm 1 bên trái
              ↑
          cặp phải: 3 ≠ 4, không cái nào bằng 1  →  LỖI ❌

Ví dụ 3:  (4, 3)  với  (4, 1)
          cặp phải: 3 với 1 → kéo giãn 1 thành 3
          cặp trái: 4 = 4   → kết quả (4, 3) ✅     (mỗi HÀNG trừ/chia cho một số
riêng)
```

Ví dụ 2 là lỗi kinh điển: muốn trừ mỗi hàng cho một số nhưng quên đổi `(4,)` thành `(4, 1)`. Cách sửa ở mục 6.15.

5.5. View (cửa sổ) và Copy (bản sao)

- **View:** một mảng mới nhưng **dùng chung bộ nhớ** với mảng gốc. Như mở một **cửa sổ** nhìn vào căn phòng: bạn sơn lại tường qua cửa sổ thì căn phòng thật bị sơn.
- **Copy:** một bản sao **độc lập**. Như chụp ảnh căn phòng: vẽ lên ảnh không ảnh hưởng căn phòng.

Thao tác	Trả về	Ví dụ
Cắt lát `a[1:3]`, `X[:, 0]`	**View**	Sửa nó là sửa mảng gốc
`reshape`, `.T`, `ravel` (thường)	**View**	
Chỉ số bằng list `a[[0, 2]]`	**Copy**	
Lọc điều kiện `a[a > 3]`	**Copy**	
`.copy()`, `flatten()`	**Copy**	Cách chắc chắn nhất

5.6. 📌 Thuật ngữ

Tên tiếng Anh	Nghĩa tiếng Việt	Giải thích 1 câu	Ví dụ
NumPy (Numerical Python)	Python số học	Thư viện làm toán nhanh trên mảng số.	Pandas, OpenCV dùng NumPy bên trong
ndarray	Mảng nhiều chiều	Khối số cùng kiểu, xếp thành lưới đều.	Bảng 4 căn nhà × 3 đặc trưng
shape	Hình dạng	Kích thước mảng theo từng chiều.	`(4, 3)`
dtype (data type)	Kiểu dữ liệu	Kiểu chung của mọi phần tử trong mảng.	`float64`, `uint8`
axis	Trục	Một chiều của mảng, đánh số từ 0.	`axis=0` là trục hàng
Vectorization	Vector hóa	Viết phép tính cho cả mảng thay cho vòng lặp.	`gia * 1000`
ufunc (universal function)	Hàm toàn cục	Hàm áp dụng cho từng phần tử, chạy bằng C.	`np.sqrt`, `np.exp`
Aggregation	Gộp / tổng hợp	Phép biến nhiều số thành ít số hơn.	`sum`, `mean`, `max`
Indexing / Slicing	Lấy chỉ số / cắt lát	Lấy một hoặc nhiều phần tử theo vị trí.	`a[0]`, `a[1:3]`
Fancy indexing	Lấy theo danh sách chỉ số	Lấy các vị trí bất kỳ bằng một list.	`a[[0, 3]]`
Boolean mask	Mặt nạ lọc	Mảng True/False dùng để chọn phần tử.	`a[a > 3]`

Tên tiếng Anh	Nghĩa tiếng Việt	Giải thích 1 câu	Ví dụ
Broadcasting	Lan truyền	Tự kéo giãn mảng nhỏ cho khớp mảng lớn.	`X - X.mean(axis=0)`
View / Copy	Cửa sổ / Bản sao	Dùng chung bộ nhớ / bộ nhớ riêng.	`a[:2]` / `a[:2].copy()`
Seed	Hạt giống ngẫu nhiên	Con số giúp dãy ngẫu nhiên lặp lại được.	`np.random.default_rng(42)`

6. 💻 Code từng bước (Colab)

> Chạy các ô **theo đúng thứ tự** từ trên xuống. Ô sau dùng biến của ô trước.

6.1. Import và kiểm tra phiên bản

```
import numpy as np        # Colab đã cài sẵn NumPy, chỉ cần import
print(np.__version__)     # ví dụ: 2.0.2 (bản của bạn có thể khác, không sao)
```

Nếu chạy trên máy riêng mà báo `ModuleNotFoundError`, cài bằng `pip install numpy` (bài 0.2).

6.2. Tạo mảng từ list

```
# Giá 4 căn nhà (tỷ đồng): từ list 1 chiều → mảng 1 chiều
gia_nha = np.array([2.5, 3.1, 4.0, 5.2])
print(gia_nha)
print(type(gia_nha))          # kiểu của đối tượng: numpy.ndarray

# 4 căn nhà × 3 đặc trưng: diện tích (m²), số phòng ngủ, khoảng cách tới trung tâm (km)
# Từ list lồng nhau → mảng 2 chiều. Mỗi list con là MỘT HÀNG
X = np.array([
    [50,  2, 5],
    [65,  2, 3],
    [80,  3, 7],
    [100, 3, 2],
])
print(X)
```

Kết quả mẫu:

```
[2.5 3.1 4.  5.2]
<class 'numpy.ndarray'>
[[ 50   2   5]
 [ 65   2   3]
```

```
 [ 80   3   7]
 [100   3   2]]
```

- NumPy in mảng **không có dấu phẩy** giữa các phần tử. Đó là cách nhận ra "đây là mảng NumPy, không phải list".
- `4.` chính là `4.0`. NumPy viết gọn.
- Trong ML, ma trận dữ liệu thường đặt tên là **X viết hoa**, cột đích (thứ cần dự đoán) là **y viết thường**. Mỗi **hàng** là một **mẫu (sample)**, mỗi **cột** là một **feature (đặc trưng)**.

6.3. "Căn cước" của mảng

```
print("gia_nha:", gia_nha.shape, gia_nha.ndim, gia_nha.size, gia_nha.dtype)
print("X      :", X.shape, X.ndim, X.size, X.dtype)

print(X.itemsize)     # số byte của MỘT phần tử
print(X.nbytes)       # tổng số byte = size × itemsize
print(len(X))         # len() chỉ đếm theo trục 0 (số hàng)
```

Kết quả mẫu:

```
gia_nha: (4,) 1 4 float64
X      : (4, 3) 2 12 int64
8
96
4
```

- `shape (4,)`: mảng 1 chiều có 4 phần tử. Dấu phẩy thừa để nhấn mạnh đây là **tuple 1 phần tử**.
- `size` = tổng số phần tử = 4 × 3 = 12.
- `X` toàn số nguyên nên dtype là `int64`. `gia_nha` có số lẻ nên là `float64`.

6.4. Các hàm tạo mảng có sẵn

Trong thực tế bạn hiếm khi gõ tay từng số. NumPy có sẵn nhiều "khuôn":

```
print(np.zeros(3))                # 3 số 0 (kiểu float)
print(np.ones((2, 3)))            # bảng 2×3 toàn số 1. CHÚ Ý: shape truyền bằng
tuple (2, 3)
print(np.full((2, 2), 7))         # bảng 2×2 toàn số 7
print(np.eye(3))                  # ma trận đơn vị 3×3: đường chéo là 1, còn lại
0
print(np.arange(5))               # 0, 1, 2, 3, 4 (giống range nhưng ra mảng)
print(np.arange(0, 10, 2))        # từ 0 đến TRƯỚC 10, bước nhảy 2
print(np.arange(1, 2, 0.25))      # bước nhảy có thể là số lẻ
print(np.linspace(0, 1, 5))       # 5 số CÁCH ĐỀU từ 0 đến 1, tính CẢ số 1
print(np.zeros_like(X))           # mảng 0 có cùng shape và dtype với X
```

Kết quả mẫu:

```
[0. 0. 0.]
[[1. 1. 1.]
 [1. 1. 1.]]
[[7 7]
 [7 7]]
[[1. 0. 0.]
 [0. 1. 0.]
 [0. 0. 1.]]
[0 1 2 3 4]
[0 2 4 6 8]
[1.   1.25 1.5  1.75]
[0.   0.25 0.5  0.75 1.  ]
[[0 0 0]
 [0 0 0]
 [0 0 0]
 [0 0 0]]
```

`arange` hay `linspace`?

Bạn biết...	Dùng	Ví dụ
Bước nhảy	`np.arange(start, stop, step)`	Các mức giá cách nhau 0,5 tỷ
Số lượng điểm	`np.linspace(start, stop, num)`	100 điểm để vẽ đường cong mượt (bài 1.5, 1.9)

> 💡 `np.empty(3)` cũng tạo mảng 3 phần tử, nhưng **không xóa** dữ liệu cũ trong bộ nhớ, nên chứa số "rác" ngẫu nhiên. Chỉ dùng khi bạn chắc chắn sẽ ghi đè toàn bộ. Người mới nên dùng `np.zeros`.

6.5. dtype: chọn đúng kiểu số

6.5.1. NumPy tự đoán kiểu

```
print(np.array([1, 2, 3]).dtype)          # toàn số nguyên → int64
print(np.array([1, 2, 3.5]).dtype)        # có 1 số thực → TẤT CẢ thành float64
print(np.array([1, 2, "ba"]))             # có 1 chuỗi → TẤT CẢ thành chuỗi!
print(np.array([True, False]).dtype)      # bool
```

Kết quả mẫu:

```
int64
float64
['1' '2' 'ba']
bool
```

Mảng chỉ có **một** kiểu. Nếu trộn, NumPy "nâng cấp" lên kiểu chứa được tất cả: `bool → int → float → chuỗi`. Mảng `['1' '2' 'ba']` giờ là **chữ**, không cộng trừ được nữa. Đây là lý do dữ liệu bảng có cả chữ và số nên để **Pandas** xử lý (bài 1.4).

6.5.2. Chỉ định và đổi kiểu với `astype`

```
a = np.array([1.2, 2.7, 3.9, -1.5])
print(a.astype(int))            # float → int: CẮT BỎ phần lẻ, KHÔNG làm tròn!
print(np.round(a).astype(int))  # muốn làm tròn thì round trước rồi mới đổi kiểu
print(a.astype(np.float32).dtype, a.astype(np.float32).nbytes, a.nbytes)  #
float32 tốn nửa bộ nhớ

b = np.array([1, 2, 3], dtype=np.float32)   # chỉ định dtype ngay khi tạo
print(b, b.dtype)
```

Kết quả mẫu:

```
[ 1  2  3 -1]
[ 1  3  4 -2]
float32 16 32
[1. 2. 3.] float32
```

- `2.7 → 2`, `3.9 → 3`, `-1.5 → -1`: đổi sang int là **chặt cụt về phía 0**.
- `astype` luôn trả về **mảng mới**, mảng gốc không đổi.

6.5.3. Tràn số với `uint8`: chuyện thật của ảnh

Pixel ảnh có giá trị 0–255, lưu bằng `uint8`. Thử tăng sáng ảnh bằng cách cộng 100:

```
pixel = np.array([50, 200, 250], dtype=np.uint8)   # 3 pixel: tối, sáng, rất sáng
print(pixel + 100)                                  # tràn số! 200 + 100 = 300 →
44

# Cách đúng: đổi sang kiểu lớn hơn, cộng, rồi "kẹp" vào khoảng 0..255
sang_hon = np.clip(pixel.astype(np.int16) + 100, 0, 255).astype(np.uint8)
print(sang_hon)
```

Kết quả mẫu:

```
[150  44  94]
[150 255 255]
```

`uint8` chỉ chứa được 0–255. Vượt quá thì nó "quay vòng" như đồng hồ: 300 − 256 = 44. Pixel đang **rất sáng lại thành tối**, ảnh bị lốm đốm. Bạn sẽ gặp lại lỗi này ở bài CV1.1. `np.clip(x, thấp, cao)` ép mọi giá trị vào

trong khoảng `[thấp, cao]`.

6.6. Lấy phần tử và cắt lát

6.6.1. Mảng 1 chiều

```
chỉ số dương:     0    1    2    3    4    5    6    7    8
              [  10 , 20 , 30 , 40 , 50 , 60 , 70 , 80 , 90 ]
chỉ số âm:       -9   -8   -7   -6   -5   -4   -3   -2   -1
```

```
a = np.arange(10, 100, 10)     # [10 20 30 40 50 60 70 80 90]
print(a[0], a[-1], a[-2])      # phần tử đầu, cuối, áp cuối
print(a[2:5])                  # a[start:stop]: từ vị trí 2 đến TRƯỚC vị trí 5
print(a[:3], a[6:])            # bỏ start = từ đầu; bỏ stop = đến hết
print(a[::2])                  # a[start:stop:step]: lấy cách 1 phần tử
print(a[::-1])                 # step âm: đảo ngược mảng
```

Kết quả mẫu:

```
10 90 80
[30 40 50]
[10 20 30] [70 80 90]
[10 30 50 70 90]
[90 80 70 60 50 40 30 20 10]
```

Quy tắc giống hệt list (bài 1.1). Mẹo nhớ: `a[2:5]` lấy **5 − 2 = 3** phần tử.

Gán giá trị qua chỉ số và lát cắt:

```
a[0] = 11              # sửa 1 phần tử
a[1:3] = 0             # gán CÙNG MỘT số cho cả lát cắt (list Python không làm được
thế này)
print(a)
```

Kết quả mẫu:

```
[11  0  0 40 50 60 70 80 90]
```

6.6.2. Mảng 2 chiều: `X[hàng, cột]`

```
              cột 0   cột 1   cột 2
              (m²)   (phòng)  (km)
   hàng 0  [   50       2       5  ]
   hàng 1  [   65       2       3  ]
   hàng 2  [   80       3       7  ]
   hàng 3  [  100       3       2  ]
```

```
print(X[1, 2])         # hàng 1, cột 2: khoảng cách của căn thứ hai
print(X[0])            # hàng 0: toàn bộ thông tin căn đầu tiên (viết gọn của X[0,
:])
print(X[-1])           # hàng cuối
print(X[:, 0])         # cột 0: diện tích của MỌI căn. Dấu ":" nghĩa là "lấy hết"
print(X[:, -1])        # cột cuối
print(X[1:3, :2])      # hàng 1–2, cột 0–1
```

Kết quả mẫu:

```
3
[50  2  5]
[100   3   2]
[ 50  65  80 100]
[5 3 7 2]
[[65  2]
 [80  3]]
```

> ⚠️ Viết `X[1][2]` cũng ra 3, nhưng chậm hơn và **không** dùng được cho cột (`X[:][0]` lấy **hàng** 0, không phải cột 0!). Hãy luôn dùng dấu phẩy: `X[1, 2]`, `X[:, 0]`.

Hình dung `X[1:3, :2]`:

```
              cột 0   cột 1   cột 2
   hàng 0  [   50       2       5  ]
   hàng 1  [ ┌─65─────────2─┐     3  ]
   hàng 2  [ └─80─────────3─┘     7  ]   ← hàng 1–2 (bỏ 3), cột 0–1 (bỏ 2)
   hàng 3  [  100       3       2  ]
```

6.6.3. Mảng 3 chiều: một bức ảnh

Ảnh màu là mảng `(cao, rộng, 3)`: mỗi pixel có 3 số Đỏ, Xanh lá, Xanh dương (RGB), mỗi số từ 0 đến 255.

```
anh = np.zeros((2, 3, 3), dtype=np.uint8)   # ảnh 2 pixel cao × 3 pixel rộng, toàn
màu đen
anh[0, 0] = [255, 0, 0]                     # pixel góc trên trái → đỏ
```

```
anh[1, 2] = [255, 255, 255]                 # pixel góc dưới phải → trắng
anh[:, 1] = [0, 0, 255]                     # cả cột pixel giữa → xanh dương

print(anh.shape)
print(anh[:, :, 0])                         # kênh Đỏ của mọi pixel (bảng 2×3)
```

Kết quả mẫu:

```
(2, 3, 3)
[[255   0   0]
 [  0   0 255]]
```

```
Ảnh 2×3 pixel:                Kênh Đỏ anh[:, :, 0]:
┌──────┬──────┬──────┐         [[255   0   0]
│ đỏ   │ xanh │ đen  │          [  0   0 255]]
├──────┼──────┼──────┤
│ đen  │ xanh │ trắng│        Màu trắng = (255, 255, 255) nên kênh Đỏ cũng là 255
└──────┴──────┴──────┘
```

Muốn xem ảnh thật: `import matplotlib.pyplot as plt; plt.imshow(anh); plt.show()` (sẽ học kỹ ở bài 1.5). Ảnh chụp bằng điện thoại cũng y như vậy, chỉ là `shape` lớn hơn, ví dụ `(3000, 4000, 3)`.

6.7. Fancy indexing: lấy theo danh sách vị trí

```
print(gia_nha[[0, 2]])          # lấy vị trí 0 và 2. Chú ý: HAI lớp ngoặc
print(gia_nha[[3, 3, 0]])       # lặp lại vị trí, đổi thứ tự đều được
print(X[[3, 0]])                # lấy hàng 3 rồi hàng 0
print(X[:, [0, 2]])             # lấy cột 0 và cột 2 (bỏ cột số phòng)
```

Kết quả mẫu:

```
[2.5 4. ]
[5.2 5.2 2.5]
[[100   3   2]
 [ 50   2   5]]
[[ 50   5]
 [ 65   3]
 [ 80   7]
 [100   2]]
```

Ứng dụng: sắp xếp cả bảng theo một cột.

```
thu_tu = np.argsort(gia_nha)    # VỊ TRÍ các phần tử nếu xếp tăng dần
print(thu_tu)
print(gia_nha[thu_tu])          # giá đã xếp tăng dần
print(X[thu_tu])                # cả bảng X xếp theo giá, hàng giữ nguyên vẹn
```

Kết quả mẫu:

```
[0 1 2 3]
[2.5 3.1 4.  5.2]
[[ 50   2   5]
 [ 65   2   3]
 [ 80   3   7]
 [100   3   2]]
```

6.8. Boolean mask: lọc dữ liệu bằng điều kiện

6.8.1. So sánh tạo ra mặt nạ

```
dien_tich = np.array([50, 65, 80, 100])        # diện tích 4 căn (m²)

mask = gia_nha > 3.5                           # so sánh CẢ MẢNG → mảng True/False
print(mask)
print(gia_nha[mask])                           # chỉ giữ phần tử ở vị trí True
print(dien_tich[mask])                         # dùng mặt nạ của mảng này để lọc
mảng KHÁC cùng độ dài
```

Kết quả mẫu:

```
[False False  True  True]
[4.  5.2]
[ 80 100]
```

```
gia_nha:        [ 2.5 ,  3.1 ,  4.0 ,  5.2 ]
gia_nha > 3.5:  [ False, False, True , True ]   ← cái "mặt nạ"
kết quả:                        [ 4.0 ,  5.2 ]   ← chỉ lộ ra chỗ True
```

6.8.2. Kết hợp nhiều điều kiện

Ý nghĩa	Python thường	NumPy (bắt buộc dùng)
VÀ	and	&

Ý nghĩa	Python thường	NumPy (bắt buộc dùng)
HOẶC	or	|
PHỦ ĐỊNH	not	~

```
# BẮT BUỘC có ngoặc quanh mỗi điều kiện, vì & được tính TRƯỚC dấu >
print(dien_tich[(gia_nha > 3) & (dien_tich < 90)])    # giá > 3 tỷ VÀ diện tích < 90 m²
print(dien_tich[(gia_nha < 3) | (dien_tich > 90)])    # rất rẻ HOẶC rất rộng
print(gia_nha[~(gia_nha > 3.5)])                      # KHÔNG đắt hơn 3,5 tỷ
```

Kết quả mẫu:

```
[65 80]
[ 50 100]
[2.5 3.1]
```

6.8.3. Đếm, kiểm tra, thay thế

```
print((gia_nha > 3.5).sum())          # đếm số True (True = 1, False = 0)
print((gia_nha > 3.5).mean())         # TỈ LỆ True: 50% số căn trên 3,5 tỷ
print(np.any(gia_nha > 5), np.all(gia_nha > 2))   # có ít nhất 1? tất cả đều?

so_phong = np.array([2, 2, 3, 3])
print(np.isin(so_phong, [3, 4]))      # mỗi phần tử có nằm trong danh sách [3, 4] không?

# np.where(điều kiện, giá trị nếu đúng, giá trị nếu sai)
print(np.where(gia_nha > 3.5, "Đắt", "Vừa"))

# np.where(điều kiện) chỉ 1 tham số → trả về VỊ TRÍ các phần tử True
print(np.where(gia_nha > 3.5))
```

Kết quả mẫu:

```
2
0.5
True True
[False False  True  True]
['Vừa' 'Vừa' 'Đắt' 'Đắt']
(array([2, 3]),)
```

Mẹo `mask.mean()` ra **tỉ lệ** cực kỳ hay dùng: tỉ lệ email spam, tỉ lệ dự đoán đúng (chính là **accuracy**, sẽ học ở bài 2.5).

6.8.4. Sửa dữ liệu bằng mặt nạ

```
diem = np.array([7.5, 11.0, 9.0, -1.0, 8.0])   # bảng điểm có lỗi nhập liệu: 11 và
-1
diem_sua = diem.copy()                          # làm trên bản sao để giữ dữ liệu
gốc
diem_sua[diem_sua > 10] = 10                    # điểm > 10 → 10
diem_sua[diem_sua < 0] = 0                      # điểm âm → 0
print(diem_sua)
print(np.clip(diem, 0, 10))                     # cách ngắn hơn: kẹp vào [0, 10]
```

Kết quả mẫu:

```
[ 7.5 10.   9.   0.   8. ]
[ 7.5 10.   9.   0.   8. ]
```

6.9. Phép toán vector hóa

6.9.1. Phép toán số học

```
print(gia_nha * 1000)                       # đổi tỷ → triệu: nhân CẢ MẢNG với 1000
gia_m2 = gia_nha * 1000 / dien_tich         # chia TỪNG CẶP phần tử tương ứng
print(gia_m2.round(1))                      # giá mỗi m² (triệu đồng), làm tròn 1 số
lẻ

a = np.array([7, 8, 9])
b = np.array([2, 3, 4])
print(a + b, a - b, a * b, a / b)           # cộng, trừ, nhân, chia từng cặp
print(a // b, a % b, a ** 2)                # chia lấy nguyên, chia lấy dư, lũy thừa
print(a > b, a == 8)                        # so sánh cũng theo từng cặp
```

Kết quả mẫu:

```
[2500. 3100. 4000. 5200.]
[50.  47.7 50.  52. ]
[ 9 11 13] [5 5 5] [14 24 36] [3.5        2.66666667 2.25      ]
[3 2 2] [1 2 1] [49 64 81]
[ True  True  True] [False  True False]
```

Hình dung phép chia từng cặp:

```
gia_nha*1000:  [ 2500 , 3100 , 4000 , 5200 ]
                   ÷      ÷      ÷      ÷
dien_tich:     [   50 ,   65 ,   80 ,  100 ]
                   =      =      =      =
gia_m2:        [   50 , 47.7 ,   50 ,   52 ]
```

> ⚠️ **List khác mảng:** `[1, 2] * 2` là `[1, 2, 1, 2]` (nhân bản list), còn `np.array([1, 2]) * 2` là `[2 4]` (nhân từng số). `[1, 2] + [3, 4]` là `[1, 2, 3, 4]` (nối list), còn với mảng là `[4 6]`.

6.9.2. Hàm toán học (ufunc)

```
x = np.array([1, 4, 9, 16])
print(np.sqrt(x))                  # căn bậc hai từng phần tử
print(np.exp([0, 1, 2]).round(3))  # e mũ x (e ≈ 2,718) – gặp lại ở sigmoid, bài 2.4
print(np.log([1, np.e, 100]).round(3))   # logarit tự nhiên ln(x)
print(np.abs([-3, 2, -1]))         # trị tuyệt đối
print(np.round([2.345, 2.355], 2), np.floor(2.7), np.ceil(2.1))   # làm tròn, làm tròn xuống, lên

z = np.array([-2.0, -0.5, 0.0, 1.5, 3.0])
print(np.maximum(z, 0))            # so từng phần tử với 0, giữ số lớn hơn
```

Kết quả mẫu:

```
[1. 2. 3. 4.]
[1.    2.718 7.389]
[0.    1.    4.605]
[3 2 1]
[2.35 2.36] 2.0 3.0
[0.  0.  0.  1.5 3. ]
```

- `np.maximum(z, 0)` chính là hàm kích hoạt **ReLU**, hàm phổ biến nhất trong mạng nơ-ron (sẽ học kỹ ở bài 3.3). Bạn vừa viết nó trong 1 dòng.
- `np.round(2.345, 2)` có thể ra `2.35` hoặc `2.34` tùy cách máy lưu số lẻ (mục 6.19). Đừng ngạc nhiên.
- `np.max(a)` khác `np.maximum(a, b)`: `max` tìm **một** số lớn nhất của cả mảng; `maximum` so **từng cặp** của hai mảng.

6.10. Thống kê và tính theo trục (axis)

6.10.1. Thống kê trên mảng 1 chiều

```
print(gia_nha.sum(), gia_nha.mean(), np.median(gia_nha))   # tổng, trung bình, trung vị
```

```
print(gia_nha.min(), gia_nha.max())                    # nhỏ nhất, lớn nhất
print(gia_nha.argmin(), gia_nha.argmax())              # VỊ TRÍ của số nhỏ
nhất, lớn nhất
print(gia_nha.std().round(3), gia_nha.var().round(3))  # độ lệch chuẩn,
phương sai (bài 1.7)
print(np.percentile(gia_nha, [25, 50, 75]))            # phân vị 25%, 50%,
75% (bài 1.7)
print(gia_nha.cumsum())                                # tổng cộng dồn
```

Kết quả mẫu:

```
14.8 3.7 3.55
2.5 5.2
0 3
1.017 1.035
[2.95 3.55 4.3 ]
[ 2.5  5.6  9.6 14.8]
```

- `argmax()` trả về `3` nghĩa là căn ở vị trí 3 (căn thứ tư) đắt nhất. Bạn sẽ dùng `argmax` rất nhiều: mô hình phân loại trả về xác suất cho từng lớp, `argmax` chọn lớp có xác suất cao nhất (sẽ gặp lại ở bài 3.13).
- `median` không có dạng method `.median()`, phải viết `np.median(a)`.
- `cumsum` cộng dồn: `2.5`, `2.5+3.1=5.6`, `5.6+4.0=9.6`, `9.6+5.2=14.8`. Rất hay dùng cho doanh thu lũy kế.

6.10.2. Tính theo trục với mảng 2 chiều

```
print(X.mean(axis=0))              # trung bình MỖI CỘT (mỗi đặc trưng)
print(X.max(axis=0))               # lớn nhất mỗi cột
print(X.sum(axis=1))               # tổng mỗi hàng (ở đây ít ý nghĩa, chỉ để minh
họa)
print(X.mean())                    # không ghi axis → gộp TẤT CẢ thành 1 số
```

Kết quả mẫu:

```
[73.75  2.5   4.25]
[100   3   7]
[ 57  70  90 105]
26.833333333333332
```

Đọc kết quả: diện tích trung bình 73,75 m², trung bình 2,5 phòng, trung bình cách trung tâm 4,25 km.

6.10.3. Ví dụ thực tế: doanh thu chuỗi cà phê

Chuỗi cà phê có 4 chi nhánh, theo dõi doanh thu 7 ngày (triệu đồng):

```
chi_nhanh = np.array(["Quận 1", "Quận 3", "Bình Thạnh", "Thủ Đức"])
ngay = np.array(["T2", "T3", "T4", "T5", "T6", "T7", "CN"])
doanh_thu = np.array([
    [12, 15, 14, 16, 22, 30, 28],    # Quận 1
    [ 9, 10, 15, 10, 15, 20, 18],    # Quận 3
    [11, 12, 10, 13, 17, 25, 31],    # Bình Thạnh
    [ 6,  7,  8,  7, 10, 14, 15],    # Thủ Đức
])
print(doanh_thu.shape)                          # (4 chi nhánh, 7 ngày)

tong_cn = doanh_thu.sum(axis=1)                 # gộp 7 ngày → tổng mỗi chi nhánh
print(dict(zip(chi_nhanh.tolist(), tong_cn.tolist())))   # tolist(): đổi mảng về
list Python để in gọn
print("Chi nhánh mạnh nhất:", chi_nhanh[tong_cn.argmax()])

tong_ngay = doanh_thu.sum(axis=0)               # gộp 4 chi nhánh → tổng mỗi ngày
print(tong_ngay)
print("Ngày đông khách nhất:", ngay[tong_ngay.argmax()])

# Mỗi ngày, chi nhánh nào bán nhiều nhất? argmax theo axis=0 cho 7 vị trí
print(chi_nhanh[doanh_thu.argmax(axis=0)])

# Có bao nhiêu ngày mỗi chi nhánh đạt trên 15 triệu?
print((doanh_thu > 15).sum(axis=1))
```

Kết quả mẫu:

```
(4, 7)
{'Quận 1': 137, 'Quận 3': 97, 'Bình Thạnh': 119, 'Thủ Đức': 67}
Chi nhánh mạnh nhất: Quận 1
[38 44 47 46 64 89 92]
Ngày đông khách nhất: CN
['Quận 1' 'Quận 1' 'Quận 3' 'Quận 1' 'Quận 1' 'Quận 1' 'Bình Thạnh']
[4 2 3 0]
```

Chỉ vài dòng, không một vòng lặp, ta trả lời được 5 câu hỏi kinh doanh. Để ý `chi_nhanh[doanh_thu.argmax(axis=0)]`: kết hợp `argmax` theo trục với fancy indexing (mục 6.7).

6.10.4. `keepdims`: giữ lại trục để tính tiếp

Câu hỏi: mỗi ngày đóng góp bao nhiêu % doanh thu tuần **của từng chi nhánh**?

```
tong_moi_cn = doanh_thu.sum(axis=1, keepdims=True)   # giữ shape (4, 1) thay vì
(4,)
print(tong_moi_cn.shape)
ti_le = doanh_thu / tong_moi_cn * 100               # (4, 7) chia (4, 1) →
broadcasting theo hàng
print(ti_le.round(1))
```

```
print(ti_le.sum(axis=1).round(1))                    # kiểm tra: mỗi hàng cộng lại
đúng 100%
```

Kết quả mẫu:

```
(4, 1)
[[ 8.8 10.9 10.2 11.7 16.1 21.9 20.4]
 [ 9.3 10.3 15.5 10.3 15.5 20.6 18.6]
 [ 9.2 10.1  8.4 10.9 14.3 21.  26.1]
 [ 9.  10.4 11.9 10.4 14.9 20.9 22.4]]
[100. 100. 100. 100.]
```

Không có `keepdims=True` thì `tong_moi_cn` có shape `(4,)` và phép chia báo lỗi (ví dụ 2 ở mục 5.4). Cuối tuần (T7, CN) chiếm khoảng 40–47% doanh thu: gợi ý tăng nhân viên vào cuối tuần.

6.10.5. Dữ liệu thiếu `np.nan`

```
nhiet_do = np.array([30.5, 31.0, np.nan, 29.5])   # cảm biến bị mất 1 lần đo
print(nhiet_do.mean())                            # nan "lây" sang mọi phép tính!
print(np.nanmean(nhiet_do), np.nanmax(nhiet_do))  # các hàm nan... bỏ qua giá trị
thiếu
print(np.isnan(nhiet_do), np.isnan(nhiet_do).sum())   # tìm và đếm chỗ thiếu
```

Kết quả mẫu:

```
nan
30.333333333333332 31.0
[False False  True False] 1
```

NaN – Not a Number là ký hiệu "thiếu dữ liệu". Chỉ một ô NaN cũng làm `mean()` ra NaN. Pandas xử lý NaN tiện hơn nhiều (bài 1.4, 1.6).

6.11. Sắp xếp, tìm giá trị duy nhất

```
print(np.sort(gia_nha))              # trả về mảng MỚI đã xếp tăng dần
print(np.sort(gia_nha)[::-1])        # giảm dần: xếp tăng rồi đảo ngược

# Top 2 căn đắt nhất: lấy 2 vị trí cuối của argsort, đảo ngược
top2 = np.argsort(gia_nha)[-2:][::-1]
print(top2, gia_nha[top2])

# Đánh giá sao của 12 bình luận trên một app giao đồ ăn
sao = np.array([5, 4, 5, 3, 5, 1, 4, 5, 2, 5, 4, 5])
gia_tri, so_lan = np.unique(sao, return_counts=True)    # giá trị khác nhau và số
```

```
lần xuất hiện
print(gia_tri, so_lan)
for g, s in zip(gia_tri, so_lan):
    print(f"{g}★ {'▉' * s} {s}")                    # vẽ "biểu đồ" bằng ký
tự
```

Kết quả mẫu:

```
[2.5 3.1 4.  5.2]
[5.2 4.  3.1 2.5]
[3 2] [5.2 4. ]
[1 2 3 4 5] [1 1 1 3 6]
1★ ▉ 1
2★ ▉ 1
3★ ▉ 1
4★ ▉▉▉ 3
5★ ▉▉▉▉▉▉ 6
```

`X.sort()` (method) sắp xếp **ngay trên mảng gốc** và không trả về gì. `np.sort(X)` trả về **bản mới**. Người mới hay viết `a = a.sort()` rồi nhận được `None`.

Sắp xếp mảng 2 chiều theo trục:

```
B = np.array([[3, 1, 2],
              [9, 7, 8]])
print(np.sort(B, axis=1))     # xếp trong TỪNG HÀNG
print(np.sort(B, axis=0))     # xếp trong TỪNG CỘT
```

Kết quả mẫu:

```
[[1 2 3]
 [7 8 9]]
[[3 1 2]
 [9 7 8]]
```

6.12. Đổi hình dạng: reshape, transpose, thêm/bớt trục

6.12.1. `reshape`

```
a = np.arange(6)                  # [0 1 2 3 4 5], shape (6,)
print(a.reshape(2, 3))            # sắp lại thành 2 hàng × 3 cột (điền theo từng
hàng)
print(a.reshape(3, 2))            # 3 hàng × 2 cột
```

```
print(a.reshape(-1, 1).shape)     # -1 = "NumPy tự tính". 1 cột → 6 hàng
print(a.reshape(1, -1).shape)     # 1 hàng → 6 cột
```

Kết quả mẫu:

```
[[0 1 2]
 [3 4 5]]
[[0 1]
 [2 3]
 [4 5]]
(6, 1)
(1, 6)
```

```
a = [0 1 2 3 4 5]

reshape(2, 3):        reshape(3, 2):       reshape(-1, 1):
[[0 1 2]              [[0 1]               [[0]
 [3 4 5]]              [2 3]                [1]
                       [4 5]]               [2]
                                            ...
                                            [5]]
```

Quy tắc: tổng số phần tử không đổi. 6 phần tử reshape thành `(2, 3)`, `(3, 2)`, `(6, 1)` được, nhưng `(4, 2)` = 8 thì báo lỗi.

Vì sao cần? Scikit-learn yêu cầu X luôn là **bảng 2 chiều**, kể cả khi chỉ có 1 đặc trưng. Nếu bạn đưa mảng 1 chiều `dien_tich` vào, nó báo lỗi `Expected 2D array, got 1D array instead`. Cách sửa: `dien_tich.reshape(-1, 1)`. Bạn sẽ gặp lỗi này ở bài 2.2.

6.12.2. Duỗi phẳng, chuyển vị

```
M = np.array([[1, 2, 3],
              [4, 5, 6]])
print(M.ravel())         # duỗi phẳng thành 1 chiều (thường là view)
print(M.flatten())       # duỗi phẳng, LUÔN là bản sao
print(M.T)               # chuyển vị (transpose): hàng thành cột, cột thành hàng
print(M.shape, M.T.shape)
```

Kết quả mẫu:

```
[1 2 3 4 5 6]
[1 2 3 4 5 6]
[[1 4]
 [2 5]
```

```
 [3 6]]
(2, 3) (3, 2)
```

Duỗi phẳng rất hay dùng trong CV: ảnh chữ số viết tay 28×28 pixel được duỗi thành vector 784 số để đưa vào mạng nơ-ron (bài 3.7).

6.12.3. Thêm và bớt trục

```
v = np.array([1, 2, 3])                 # shape (3,)
print(v[np.newaxis, :].shape)           # thêm trục ở đầu → (1, 3): 1 hàng
print(v[:, np.newaxis].shape)           # thêm trục ở cuối → (3, 1): 1 cột
print(np.expand_dims(v, axis=0).shape)  # cách viết khác của v[np.newaxis, :]

w = np.zeros((1, 3, 1))
print(np.squeeze(w).shape)              # bỏ MỌI trục có kích thước 1 → (3,)
```

Kết quả mẫu:

```
(1, 3)
(3, 1)
(1, 3)
(3,)
```

Trong Deep Learning, mô hình nhận một **lô** ảnh `(số ảnh, cao, rộng, kênh)`. Muốn dự đoán **một** ảnh `(cao, rộng, kênh)` phải thêm trục: `anh[np.newaxis]` → `(1, cao, rộng, kênh)`.

6.13. Ghép và tách mảng

```
can_moi = np.array([[70, 2, 4]])          # 1 căn mới, shape (1, 3)
X_them = np.vstack([X, can_moi])          # vstack: chồng DỌC → thêm hàng
print(X_them.shape)

gia_cot = gia_nha.reshape(-1, 1)          # giá thành 1 cột, shape (4, 1)
bang = np.hstack([X, gia_cot])            # hstack: ghép NGANG → thêm cột
print(bang)

print(np.concatenate([X, X], axis=0).shape)   # concatenate: chọn trục để ghép
print(np.column_stack([dien_tich, so_phong])) # ghép nhiều mảng 1 chiều thành các
cột
```

Kết quả mẫu:

```
(5, 3)
[[ 50.    2.    5.    2.5]
```

```
 [ 65.    2.    3.    3.1]
 [ 80.    3.    7.    4. ]
 [100.    3.    2.    5.2]]
(8, 3)
[[ 50   2]
 [ 65   2]
 [ 80   3]
 [100   3]]
```

```
vstack (thêm hàng):          hstack (thêm cột):
 [ 50  2  5]                  [ 50  2  5 | 2.5]
 [ 65  2  3]                  [ 65  2  3 | 3.1]
 [ 80  3  7]                  [ 80  3  7 | 4.0]
 [100  3  2]                  [100  3  2 | 5.2]
 ──────────
 [ 70  2  4]  ← căn mới
```

Để ý: `bang` có cả số nguyên và số thực nên **toàn bộ** thành `float64` (mục 6.5.1).

Tách mảng:

```
du_lieu = np.arange(10)
train, test = np.split(du_lieu, [8])       # cắt tại vị trí 8: 8 phần tử đầu, 2
phần tử cuối
print(train, test)
print(np.array_split(du_lieu, 3))          # chia thành 3 phần gần bằng nhau
(không cần chia hết)
```

Kết quả mẫu:

```
[0 1 2 3 4 5 6 7] [8 9]
[array([0, 1, 2, 3]), array([4, 5, 6]), array([7, 8, 9])]
```

Đây là ý tưởng đầu tiên của việc chia dữ liệu **train/test** (sẽ học kỹ ở bài 1.11). Trong thực tế phải **xáo trộn** trước khi cắt (mục 6.17).

6.14. Bài tập nhanh giữa giờ (5 phút)

Không chạy code, hãy đoán:

1. `np.arange(12).reshape(3, 4)[1]` ra gì?
2. `np.arange(12).reshape(3, 4)[:, 1]` ra gì?
3. `np.arange(12).reshape(3, 4).sum(axis=0).shape` là gì?

Sau đó chạy để kiểm tra:

```
T = np.arange(12).reshape(3, 4)
print(T)
print(T[1], T[:, 1], T.sum(axis=0).shape)
```

Kết quả mẫu:

```
[[ 0  1  2  3]
 [ 4  5  6  7]
 [ 8  9 10 11]]
[4 5 6 7] [1 5 9] (4,)
```

6.15. Broadcasting

6.15.1. Với một con số

`gia_nha * 1000` ở mục 6.9: một mảng 4 phần tử nhân với **một số**. NumPy tự "kéo giãn" con số 1000 thành `[1000, 1000, 1000, 1000]`. Đó là broadcasting đơn giản nhất.

6.15.2. Bảng với một hàng: chuẩn hóa dữ liệu

```
X (4×3)          trừ     X.mean(axis=0) (3,)  →  được kéo giãn thành (4×3)
[ 50  2  5]              [73.75 2.5 4.25]        [73.75 2.5 4.25]
[ 65  2  3]                                      [73.75 2.5 4.25]
[ 80  3  7]                                      [73.75 2.5 4.25]
[100  3  2]                                      [73.75 2.5 4.25]
```

```
X_lech = X - X.mean(axis=0)        # mỗi giá trị trừ đi trung bình của CỘT chứa nó
print(X_lech)

# Chuẩn hóa đầy đủ: trừ trung bình rồi chia độ lệch chuẩn (cũng theo cột)
X_chuan = (X - X.mean(axis=0)) / X.std(axis=0)
print(X_chuan.round(2))
print(X_chuan.mean(axis=0).round(2), X_chuan.std(axis=0))   # kiểm tra: TB ≈ 0,
ĐLC = 1
```

Kết quả mẫu:

```
[[-23.75  -0.5    0.75]
 [ -8.75  -0.5   -1.25]
 [  6.25   0.5    2.75]
 [ 26.25   0.5   -2.25]]
[[-1.28 -1.    0.39]
 [-0.47 -1.   -0.65]
```

```
 [ 0.34  1.    1.43]
 [ 1.42  1.   -1.17]]
[0. 0. 0.] [1. 1. 1.]
```

Đọc kết quả: căn đầu tiên nhỏ hơn diện tích trung bình 23,75 m^2. Sau khi chuẩn hóa, ba cột có đơn vị khác hẳn nhau (m^2, phòng, km) giờ cùng một "thang đo". Bước này cực kỳ quan trọng trong ML (sẽ học kỹ ở bài 1.11). Nếu đôi khi bạn thấy `-0.` trong kết quả: đó là một số **cực nhỏ âm** (ví dụ −0,0000000000000001) được làm tròn, không phải lỗi.

6.15.3. Bảng với một cột

Trừ mỗi **hàng** cho một số riêng: cần mảng shape `(4, 1)`.

```
tb_hang = X.mean(axis=1)                        # shape (4,)
try:
    X - tb_hang                                 # (4, 3) với (4,) → lỗi
except ValueError as e:
    print("Lỗi:", e)

print(X - tb_hang.reshape(-1, 1))               # (4, 3) với (4, 1) → OK
print(X - X.mean(axis=1, keepdims=True))        # cách gọn hơn, cùng kết quả
```

Kết quả mẫu:

```
Lỗi: operands could not be broadcast together with shapes (4,3) (4,)
[[ 31.         -17.         -14.        ]
 [ 41.66666667 -21.33333333 -20.33333333]
 [ 50.         -27.         -23.        ]
 [ 65.         -32.         -33.        ]]
[[ 31.         -17.         -14.        ]
 [ 41.66666667 -21.33333333 -20.33333333]
 [ 50.         -27.         -23.        ]
 [ 65.         -32.         -33.        ]]
```

(`try/except` đã học ở bài 1.2: bắt lỗi để chương trình không dừng.)

6.15.4. Cột với hàng: tạo cả bảng

```
hang = np.arange(1, 4).reshape(-1, 1)   # shape (3, 1): [[1], [2], [3]]
cot = np.arange(1, 6)                   # shape (5,)  : [1 2 3 4 5]
print(hang * cot)                       # (3, 1) × (5,) → (3, 5): bảng cửu chương
mini
```

Kết quả mẫu:

```
[[ 1  2  3  4  5]
 [ 2  4  6  8 10]
 [ 3  6  9 12 15]]
```

```
            cot:  1   2   3   4   5
hang: 1  →       [ 1   2   3   4   5 ]
      2  →       [ 2   4   6   8  10 ]
      3  →       [ 3   6   9  12  15 ]
```

Kỹ thuật này dùng để tính **khoảng cách giữa mọi cặp điểm** cùng lúc, nền tảng của KNN (bài 2.7) và K-Means (bài 2.13). Bạn sẽ tự làm ở Thực hành bài 3.

6.16. View và Copy

```
gia_nha = np.array([2.5, 3.1, 4.0, 5.2])   # tạo lại cho sạch

b = gia_nha[:2]           # b là CỬA SỔ nhìn vào gia_nha, KHÔNG phải bản sao
b[0] = 999
print(gia_nha)            # gia_nha cũng bị đổi!
print(np.shares_memory(b, gia_nha))   # kiểm tra: hai mảng có chung bộ nhớ không?

gia_nha = np.array([2.5, 3.1, 4.0, 5.2])
c = gia_nha[:2].copy()    # bản sao ĐỘC LẬP
c[0] = 999
print(gia_nha)            # không bị ảnh hưởng

d = gia_nha[gia_nha > 3]  # lọc điều kiện → luôn là bản sao
d[0] = 0
print(gia_nha, np.shares_memory(d, gia_nha))
```

Kết quả mẫu:

```
[999.    3.1   4.    5.2]
True
[2.5 3.1 4.  5.2]
[2.5 3.1 4.  5.2] False
```

Vì sao NumPy làm vậy? Với ảnh 3000×4000 pixel, nếu mỗi lần cắt một vùng lại sao chép thì rất tốn bộ nhớ và chậm. View giúp nhanh, nhưng bạn phải **chủ động** `.copy()` khi muốn sửa mà không ảnh hưởng dữ liệu gốc.

> Giống câu 1 mini-game bài 1.1: hai cái tên, một đồ vật.

6.17. Số ngẫu nhiên có thể lặp lại

```
rng = np.random.default_rng(42)        # bộ sinh số ngẫu nhiên; 42 là "hạt giống"
(seed)

print(rng.integers(1, 7, size=5))      # 5 lần tung xúc xắc: số nguyên từ 1 đến 6
(không gồm 7)
print(rng.random(3).round(3))          # 3 số thực ngẫu nhiên trong [0, 1)
print(rng.uniform(5, 10, size=3).round(2))     # 3 số thực đều trong [5, 10)
print(rng.normal(loc=3.5, scale=0.5, size=3).round(2))   # 3 số quanh 3.5 theo
phân phối chuẩn (bài 1.10)
print(rng.choice(["Phở", "Bún chả", "Cơm tấm"], size=4))  # chọn ngẫu nhiên từ
danh sách
print(rng.choice(["Spam", "Thường"], size=6, p=[0.2, 0.8]))  # có xác suất: 20%
Spam
```

Kết quả mẫu (cùng seed 42 sẽ ra đúng như vậy):

```
[1 5 4 3 3]
[0.697 0.094 0.976]
[8.81 8.93 5.64]
[3.07 3.94 3.89]
['Cơm tấm' 'Cơm tấm' 'Bún chả' 'Bún chả']
['Thường' 'Thường' 'Thường' 'Spam' 'Thường' 'Thường']
```

Chạy lại ô trên, bạn nhận **đúng** các số đó, vì ô bắt đầu bằng `default_rng(42)`. Trong ML, luôn đặt seed để **người khác chạy lại ra đúng kết quả của bạn**.

Xáo trộn dữ liệu trước khi chia train/test:

```
rng = np.random.default_rng(0)
vi_tri = rng.permutation(len(X))       # hoán vị ngẫu nhiên các chỉ số 0..3
print(vi_tri)
X_tron, gia_tron = X[vi_tri], gia_nha[vi_tri]   # dùng CÙNG thứ tự cho X và giá để
không lệch cặp
print(X_tron)
print(gia_tron)
```

Kết quả mẫu:

```
[2 0 1 3]
[[ 80   3   7]
 [ 50   2   5]
 [ 65   2   3]
 [100   3   2]]
[4.  2.5 3.1 5.2]
```

> 💡 Bạn sẽ thấy nhiều tài liệu cũ dùng `np.random.seed(42)` và `np.random.rand(...)`. Cách đó vẫn chạy, nhưng NumPy khuyên dùng `default_rng` cho code mới.

6.18. Nhân ma trận: mô hình dự đoán đầu tiên của bạn

Giả sử một "chuyên gia" nói giá nhà (tỷ) ≈ 0,05 × diện tích + 0,3 × số phòng − 0,1 × khoảng cách. Tính giá dự đoán cho cả 4 căn:

```
w = np.array([0.05, 0.3, -0.1])        # trọng số cho 3 đặc trưng

# Cách 1: nhân từng cặp rồi cộng theo hàng
du_doan_1 = (X * w).sum(axis=1)
# Cách 2: phép nhân ma trận @ (sẽ học kỹ ở bài 1.8)
du_doan_2 = X @ w

print(du_doan_1, du_doan_2)
print("Sai lệch so với giá thật:", (du_doan_2 - gia_nha).round(2))
```

Kết quả mẫu:

```
[2.6  3.55 4.2  5.7 ] [2.6  3.55 4.2  5.7 ]
Sai lệch so với giá thật: [0.1  0.45 0.2  0.5 ]
```

Tính tay căn đầu tiên: 0,05 × 50 + 0,3 × 2 − 0,1 × 5 = 2,5 + 0,6 − 0,5 = **2,6** tỷ. Giá thật 2,5 tỷ, lệch 0,1.

Đây chính là dạng của **Linear Regression (hồi quy tuyến tính)** (bài 2.2): dự đoán = `X @ w`. Việc của "học máy" là **tự tìm** bộ `w` tốt nhất thay vì nhờ chuyên gia đoán.

6.19. Số thực không chính xác tuyệt đối

```
print(0.1 + 0.2, 0.1 + 0.2 == 0.3, np.isclose(0.1 + 0.2, 0.3))
print(np.allclose([0.1 + 0.2, 1.0], [0.3, 1.0]))     # so sánh cả mảng
print(np.nan == np.nan)                               # NaN không bằng chính nó!
```

Kết quả mẫu:

```
0.30000000000000004 False True
True
False
```

Máy tính lưu số thập phân dạng nhị phân nên có sai số rất nhỏ. **Không so sánh số thực bằng** `==`, hãy dùng `np.isclose()` (từng phần tử) hoặc `np.allclose()` (cả mảng). Muốn tìm NaN, dùng `np.isnan()`, không dùng `== np.nan`.

6.20. Lưu và đọc mảng

```
np.save("X.npy", X)                          # định dạng nhị phân của NumPy: nhanh,
giữ nguyên dtype, shape
X_doc = np.load("X.npy")
print(X_doc.shape, np.array_equal(X, X_doc))

np.savetxt("X.csv", X, delimiter=",", fmt="%d", header="dien_tich,phong,km",
comments="")
print(open("X.csv").read())                  # xem nội dung file CSV vừa ghi
X_csv = np.loadtxt("X.csv", delimiter=",", skiprows=1)   # skiprows=1: bỏ dòng
tiêu đề
print(X_csv)
```

Kết quả mẫu:

```
(4, 3) True
dien_tich,phong,km
50,2,5
65,2,3
80,3,7
100,3,2

[[ 50.   2.   5.]
 [ 65.   2.   3.]
 [ 80.   3.   7.]
 [100.   3.   2.]]
```

- `.npy` dùng để lưu kết quả trung gian (ví dụ đặc trưng ảnh đã trích xuất, bài CV2).
- CSV đọc bằng NumPy chỉ hợp với file **toàn số**. File có cả chữ, có ô trống → dùng Pandas (bài 1.4).

---

7. ⚠️ Lỗi thường gặp và hiểu lầm phổ biến

Lỗi 1: Dùng `and` / `or` với mảng

```
try:
    gia_nha[(gia_nha > 3) and (gia_nha < 5)]
except ValueError as e:
    print("Lỗi:", e)
print(gia_nha[(gia_nha > 3) & (gia_nha < 5)])     # sửa: dùng & và có ngoặc
```

Kết quả mẫu:

```
Lỗi: The truth value of an array with more than one element is ambiguous. Use
a.any() or a.all()
```

```
[3.1 4. ]
```

- **Nhận biết:** thông báo `The truth value of an array ... is ambiguous` (giá trị đúng/sai của mảng là mơ hồ).
- **Vì sao:** `and` hỏi "cả mảng đúng hay sai?", NumPy không biết trả lời một câu cho 4 phần tử.
- **Sửa:** `&`, `|`, `~` và **ngoặc quanh mỗi điều kiện**. Quên ngoặc `gia_nha > 3 & gia_nha < 5` cũng gây lỗi này, vì `&` được tính trước `>`.

Lỗi 2: Shape không khớp khi broadcasting

```
try:
    X / X.sum(axis=1)                  # (4, 3) chia (4,)
except ValueError as e:
    print("Lỗi:", e)
print((X / X.sum(axis=1, keepdims=True)).round(2))   # sửa: keepdims → (4, 1)
```

Kết quả mẫu:

```
Lỗi: operands could not be broadcast together with shapes (4,3) (4,)
[[0.88 0.04 0.09]
 [0.93 0.03 0.04]
 [0.89 0.03 0.08]
 [0.95 0.03 0.02]]
```

- **Nhận biết:** `operands could not be broadcast together with shapes (4,3) (4,)`.
- **Sửa: in `shape` của cả hai mảng** ra trước tiên. Dùng `keepdims=True` hoặc `.reshape(-1, 1)`.
- **Nguy hiểm hơn:** nếu bảng là vuông (ví dụ 3×3), phép tính **không báo lỗi** mà âm thầm chia theo cột thay vì theo hàng. Luôn kiểm tra kết quả bằng một ô tính tay.

Lỗi 3: Sửa lát cắt làm hỏng dữ liệu gốc

```
du_lieu_goc = np.array([10, 20, 30, 40])
mau = du_lieu_goc[:2]          # tưởng là "lấy mẫu ra để thử nghiệm"
mau[:] = 0                     # thử nghiệm trên mẫu...
print(du_lieu_goc)             # ...dữ liệu gốc cũng mất!
```

Kết quả mẫu:

```
[ 0  0 30 40]
```

- **Nhận biết:** số liệu "tự nhiên" thay đổi ở những ô bạn không hề đụng tới.
- **Sửa:** `mau = du_lieu_goc[:2].copy()`. Quy tắc an toàn: **lát cắt mà định sửa thì `.copy()`**.

Lỗi 4: Gán số thực vào mảng số nguyên

```
diem = np.array([7, 8, 9])     # dtype int64 vì toàn số nguyên
diem[0] = 7.9                  # tưởng gán 7.9...
print(diem)                    # ...nhưng bị cắt thành 7!
diem_f = np.array([7, 8, 9], dtype=float)   # sửa: khai báo float từ đầu
diem_f[0] = 7.9
print(diem_f)
```

Kết quả mẫu:

```
[7 8 9]
[7.9 8.  9. ]
```

- **Nhận biết:** số lẻ "biến mất" không báo lỗi gì.
- **Sửa:** kiểm tra `dtype` trước khi gán; tạo mảng với `dtype=float`, hoặc gõ `7.0` thay vì `7`.

Lỗi 5: Nhầm `axis`

- **Hiểu lầm:** "`axis=0` là tính theo hàng". Thực ra `axis=0` **gộp các hàng** lại, nên kết quả là **mỗi cột ra 1 số**.
- **Nhận biết:** in `shape` của kết quả. Muốn mỗi cột một số mà ra số phần tử bằng số hàng → sai trục.
- **Mẹo nhớ:** trục ghi trong `axis` là trục **biến mất**.

Lỗi 6: List lồng nhau không đều

```
try:
    np.array([[1, 2, 3], [4, 5]])     # hàng 1 có 3 số, hàng 2 có 2 số
except ValueError as e:
    print("Lỗi:", e)
```

Kết quả mẫu:

```
Lỗi: setting an array element with a sequence. The requested array has an
inhomogeneous shape after 1 dimensions. The detected shape was (2,) +
inhomogeneous part.
```

- **Nhận biết:** `inhomogeneous shape` (hình dạng không đồng nhất).
- **Sửa:** mọi hàng phải cùng độ dài. Dữ liệu thiếu thì điền `np.nan`, không bỏ trống.

---

8. 🌎 Ứng dụng trong thực tế

8.1. Tài chính: tính lợi nhuận cổ phiếu theo ngày

```
gia_cp = np.array([25.0, 25.5, 24.8, 26.0, 27.3, 26.9])   # giá đóng cửa 6 phiên
(nghìn đồng)
loi_nhuan = np.diff(gia_cp) / gia_cp[:-1] * 100           # np.diff: hiệu 2 phần
tử liên tiếp
print(loi_nhuan.round(2))                                 # % tăng/giảm mỗi phiên
print("Số phiên tăng:", (loi_nhuan > 0).sum(), "| Biến động (ĐLC):",
loi_nhuan.std().round(2), "%")
print("Lãi nếu mua phiên đầu, bán phiên cuối:", round((gia_cp[-1] / gia_cp[0] - 1)
* 100, 2), "%")
```

Kết quả mẫu:

```
[ 2.   -2.75  4.84  5.   -1.47]
Số phiên tăng: 3 | Biến động (ĐLC): 3.18 %
Lãi nếu mua phiên đầu, bán phiên cuối: 7.6 %
```

`gia_cp[:-1]` là "mọi phiên trừ phiên cuối", ghép với `np.diff` cho ra "giá phiên sau − giá phiên trước, chia giá phiên trước". Các quỹ đầu tư tính chỉ số này cho hàng nghìn mã cổ phiếu mỗi ngày, hoàn toàn bằng phép toán mảng.

8.2. Gợi ý món ăn: đo độ giống nhau giữa hai khách hàng

Mỗi khách được mô tả bằng mức thích 4 loại món (0–5): Bún/Phở, Cơm, Đồ nướng, Chè.

```
an  = np.array([5, 1, 4, 0])
binh = np.array([4, 0, 5, 1])
chi = np.array([0, 5, 1, 4])

def do_giong(u, v):
    # Cosine similarity: gần 1 = khẩu vị rất giống, gần 0 = khác hẳn (sẽ học kỹ ở
bài 1.8)
    return (u @ v) / (np.linalg.norm(u) * np.linalg.norm(v))

print("An - Bình:", do_giong(an, binh).round(2))
print("An - Chi :", do_giong(an, chi).round(2))
```

Kết quả mẫu:

```
An - Bình: 0.95
An - Chi : 0.21
```

An và Bình rất giống nhau, nên món Bình vừa đặt có thể gợi ý cho An. Đây là ý tưởng cốt lõi của hệ **gợi ý (recommendation)** ở Shopee, Netflix, GrabFood.

8.3. Thị giác máy tính: ảnh chính là mảng

Mọi thao tác chỉnh ảnh cơ bản đều là phép toán mảng:

Thao tác	Code NumPy
Lật ngang ảnh	`anh[:, ::-1]`
Cắt một vùng (crop)	`anh[100:300, 50:250]`
Ảnh xám (trung bình 3 kênh)	`anh.mean(axis=2)`
Tăng sáng an toàn	`np.clip(anh.astype(int) + 50, 0, 255).astype(np.uint8)`
Đảo màu (âm bản)	`255 - anh`

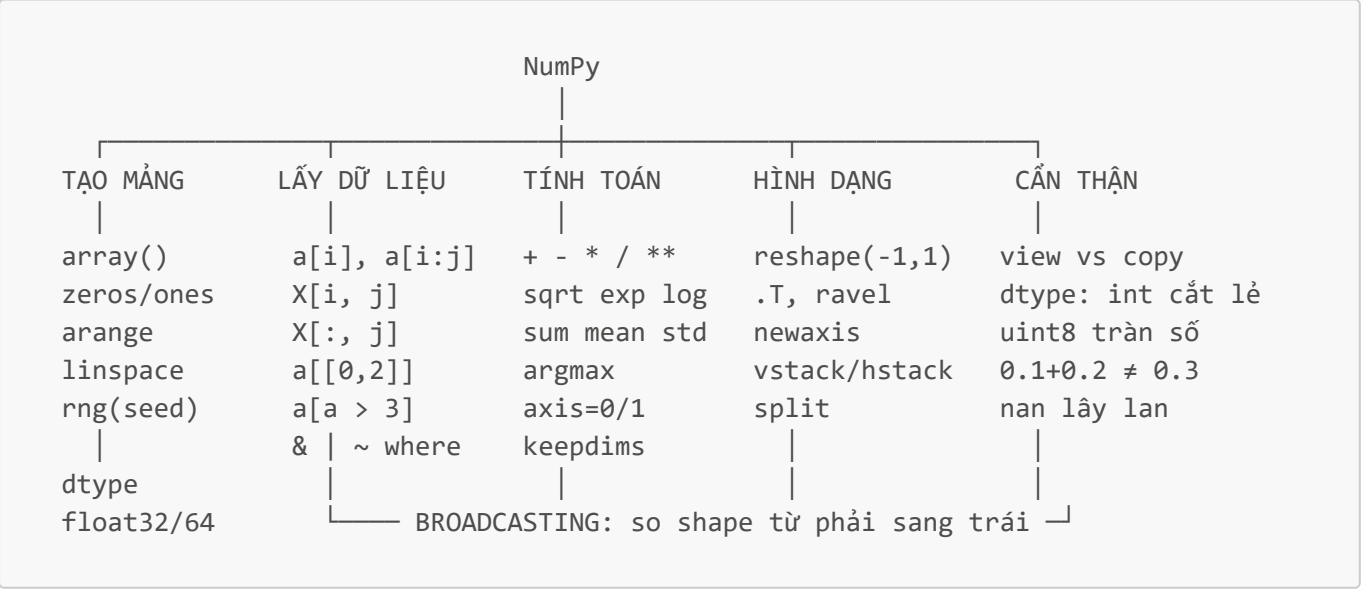
Ở nhánh CV (bài CV1.1), OpenCV đọc ảnh và trả về đúng một mảng NumPy. Mọi thứ bạn học hôm nay dùng được ngay.

---

9. 📝 Tóm tắt

1. **ndarray** là khối số cùng kiểu, nằm sát nhau trong bộ nhớ, nên tính nhanh hơn list hàng chục lần.
2. Mỗi mảng có `shape`, `ndim`, `dtype`. **In `shape` ra là bước gỡ lỗi số 1.**
3. Lấy dữ liệu: `X[hàng, cột]`, lát cắt `a:b:c`, danh sách chỉ số `a[[0, 2]]`, mặt nạ `a[a > 3]` (dùng `&`, `|`, `~`).
4. Vector hóa: viết phép tính cho cả mảng, không viết vòng lặp.
5. `axis=k` nghĩa là trục k **biến mất**: `axis=0` → mỗi cột 1 số, `axis=1` → mỗi hàng 1 số.
6. Broadcasting so `shape` từ phải sang trái: bằng nhau hoặc bằng 1 thì được. Dùng `keepdims` / `reshape(-1, 1)` khi tính theo hàng.
7. Lát cắt là **view**; muốn sửa an toàn thì `.copy()`. Luôn đặt **seed** cho số ngẫu nhiên.

Sơ đồ tư duy:



---

10. ✅ Trắc nghiệm

> Tự làm hết 10 câu trước, rồi mới xem mục 🔒 **Đáp án trắc nghiệm** ở cuối bài.

Mức 1 – Nhớ / Hiểu

Câu 1. Vì sao mảng NumPy tính nhanh hơn list Python?

- A. Vì NumPy dùng nhiều lõi CPU cho mọi phép tính.
- B. Vì phần tử cùng kiểu, nằm liền nhau trong bộ nhớ, và được xử lý bằng code C cho cả khối.
- C. Vì NumPy tự nén dữ liệu.
- D. Vì NumPy bỏ qua các số lẻ.

Câu 2. `X` có `shape (100, 5)`. Câu nào **đúng**?

- A. X có 100 cột và 5 hàng.
- B. X có 500 chiều.
- C. X có 100 mẫu, mỗi mẫu 5 đặc trưng.
- D. `len(X)` bằng 5.

Câu 3. `np.array([1, 2, 3.0]).dtype` là gì?

- A. `int64`
- B. `float64`
- C. `object`
- D. Báo lỗi vì trộn kiểu.

Câu 4. Thao tác nào trả về **view** (chung bộ nhớ với mảng gốc)?

- A. `a[a > 0]`
- B. `a[[0, 1]]`
- C. `a[1:3]`
- D. `a.copy()`

Mức 2 – Vận dụng

Câu 5. Đoạn code sau in ra gì?

```
A = np.array([[1, 2, 3],
              [4, 5, 6]])
print(A.sum(axis=0))
```

- A. `[6 15]`
- B. `[5 7 9]`
- C. `21`
- D. `[[5 7 9]]`

Câu 6. Muốn lấy các khách hàng có tuổi từ 18 đến 25 (tính cả hai đầu) trong mảng `tuoi`, cách nào đúng?

- A. `tuoi[18 <= tuoi <= 25]`
- B. `tuoi[(tuoi >= 18) and (tuoi <= 25)]`
- C. `tuoi[(tuoi >= 18) & (tuoi <= 25)]`
- D. `tuoi[tuoi >= 18 & tuoi <= 25]`

Câu 7. Đoạn code sau in ra gì?

```
a = np.array([10, 20, 30, 40, 50])
print(a[1:4][::-1])
```

- A. `[40 30 20]`
- B. `[50 40 30 20]`
- C. `[20 30 40]`
- D. `[40 30 20 10]`

Câu 8. `M` có `shape (3, 4)`, `v = M.max(axis=1)`. Muốn chia **mỗi hàng** của `M` cho số lớn nhất của hàng đó, viết thế nào?

- A. `M / v`
- B. `M / v.reshape(1, -1)`
- C. `M / v.reshape(-1, 1)`
- D. `M / v.T`

Mức 3 – Phân tích (bẫy hay gặp)

Câu 9. Đoạn code sau in ra gì?

```
p = np.array([100, 200, 250], dtype=np.uint8)
print(p + 10)
```

- A. `[110 210 260]`
- B. `[110 210 255]`
- C. `[110 210 4]`
- D. Báo lỗi tràn số.

Câu 10. Một bạn viết code chuẩn hóa từng hàng của bảng `S` shape `(3, 3)`:

```
S = np.array([[1., 2., 3.],
              [4., 5., 6.],
              [7., 8., 9.]])
S_moi = S - S.mean(axis=1)
```

Code chạy **không báo lỗi**. Nhận định nào đúng?

- A. Code đúng, mỗi hàng đã trừ đi trung bình của chính nó.
- B. Code sai: mỗi **cột** j bị trừ đi trung bình của **hàng** j, kết quả vô nghĩa nhưng không báo lỗi.
- C. Code sai và sẽ báo lỗi ở lần chạy thứ hai.
- D. Code đúng vì bảng vuông nên trục nào cũng như nhau.

---

11. 🛠️ Thực hành

Bài 1 (Cơ bản): Phân tích bảng điểm lớp học

Lớp có 6 học sinh, 3 môn Toán, Văn, Anh. Làm theo từng bước.

```
import numpy as np

ten = np.array(["An", "Bình", "Chi", "Dũng", "Em", "Giang"])
mon = np.array(["Toán", "Văn", "Anh"])
diem = np.array([
    [8.0, 6.5, 7.0],
    [5.5, 7.0, 6.0],
    [9.5, 8.0, 9.0],
    [4.0, 5.5, 3.5],
    [7.0, 9.0, 8.5],
    [6.0, 4.5, 7.5],
])

# Bước 1: In shape của bảng điểm. (Gợi ý: thuộc tính .shape)
# TODO

# Bước 2: Điểm trung bình MỖI MÔN. (Gợi ý: mean với axis nào để mỗi CỘT ra 1 số?)
# TODO

# Bước 3: Điểm trung bình MỖI HỌC SINH, làm tròn 2 số lẻ.
# TODO

# Bước 4: Ai có điểm trung bình cao nhất? (Gợi ý: argmax rồi dùng làm chỉ số cho
mảng ten)
# TODO

# Bước 5: Những ai có ít nhất một môn dưới 5? (Gợi ý: (diem < 5).any(axis=1) rồi
lọc ten)
# TODO

# Bước 6: Mỗi môn, ai cao điểm nhất? (Gợi ý: argmax theo axis=0)
# TODO

# Bước 7: Thầy cộng thêm 0.5 điểm môn Văn cho cả lớp, nhưng không ai vượt quá 10.
#         (Gợi ý: làm trên diem.copy(), cộng vào cột 1, rồi np.clip)
# TODO

# Bước 8: Điểm tổng kết có trọng số: Toán ×2, Văn ×1, Anh ×1, chia 4.
#         (Gợi ý: diem @ trong_so / 4 hoặc (diem * trong_so).sum(axis=1) / 4)
# TODO
```

Bài 2 (Trung bình): Giá nhà California chỉ bằng NumPy

Dataset: California Housing (điều tra dân số Mỹ 1990, bản rút gọn của Google Machine Learning Crash Course; gốc từ thư viện StatLib). Mỗi dòng là một **khu dân cư**. Trên Colab có sẵn ở `/content/sample_data/california_housing_train.csv`.

9 cột theo thứ tự: kinh độ, vĩ độ, tuổi nhà (trung vị), tổng số phòng, tổng phòng ngủ, dân số, số hộ, thu nhập trung vị (chục nghìn USD), giá nhà trung vị (USD).

Yêu cầu:

1. Đọc file thành mảng 2 chiều `D`. In `shape`.
2. Tách `X` = 8 cột đầu, `y` = cột cuối (giá nhà).
3. In trung bình, nhỏ nhất, lớn nhất của từng cột trong `X` (làm tròn 2 số lẻ).
4. Đếm số khu có giá **đúng bằng** giá lớn nhất. Đó là hiện tượng gì?
5. Tạo đặc trưng mới `phong_moi_ho` = tổng phòng / số hộ. Trung vị là bao nhiêu?
6. Giá trung bình của khu có thu nhập > 8 so với khu có thu nhập < 2.
7. Chuẩn hóa toàn bộ `X` theo cột (trừ trung bình, chia độ lệch chuẩn). Kiểm tra mỗi cột có trung bình ≈ 0, độ lệch chuẩn = 1.
8. Bỏ các khu bị giới hạn trần, tính lại giá trung bình. Chênh lệch bao nhiêu so với khi chưa bỏ? Điều đó nói gì?

Gợi ý: `np.loadtxt(..., delimiter=",", skiprows=1)`; dùng mặt nạ, `axis=0`, `np.median`, `np.allclose`.

```
import numpy as np

duong_dan = "/content/sample_data/california_housing_train.csv"
# Không dùng Colab? Thay bằng:
np.genfromtxt("https://download.mlcc.google.com/mledu-
datasets/california_housing_train.csv", delimiter=",", skip_header=1)

D = None          # TODO 1
X, y = None, None # TODO 2
# TODO 3 → 8
```

Bài 3 (Thử thách): Tìm căn nhà "giống nhất" không dùng vòng lặp

Một khách hàng mô tả căn nhà mơ ước: **75 m², 3 phòng ngủ, cách trung tâm 4 km**. Bạn có dữ liệu 8 căn đang bán:

```
import numpy as np
nha = np.array([
    [50, 2, 5], [65, 2, 3], [80, 3, 7], [100, 3, 2],
    [72, 3, 4], [90, 4, 6], [60, 1, 1], [78, 2, 4],
])
gia = np.array([2.5, 3.1, 4.0, 5.2, 3.6, 4.4, 3.0, 3.7])
mo_uoc = np.array([75, 3, 4])
# TODO: viết lời giải của bạn
```

Yêu cầu (không gợi ý):

1. Tìm 3 căn giống nhu cầu nhất và đề xuất mức giá hợp lý cho khách.

2. Tính **ma trận khoảng cách** 8×8 giữa mọi cặp căn nhà, rồi cho biết hai căn nào giống nhau nhất (khác nhau).
3. Toàn bộ không được dùng vòng lặp `for` / `while`.

Tiêu chí tự đánh giá:

- ☐ Đã nghĩ tới việc các cột có **đơn vị khác nhau** (m^2 lớn hơn số phòng rất nhiều) và xử lý vấn đề đó.
- ☐ Khoảng cách được tính cho **cả 8 căn** bằng một phép toán broadcasting.
- ☐ Ma trận khoảng cách có `shape (8, 8)`, đối xứng, đường chéo bằng 0, và bạn đã loại đường chéo khi tìm cặp giống nhất.
- ☐ Giải thích được kết quả bằng lời (vì sao các căn đó giống).
- ☐ Không có vòng lặp nào.

> Đây là ý tưởng của thuật toán **KNN – K-Nearest Neighbors (K láng giềng gần nhất)**, bạn sẽ học ở bài 2.7.

---

12. 🃏 Flashcard

Mặt trước	Mặt sau
`axis=0` trên bảng (4, 3) cho kết quả shape gì?	`(3,)`: trục 0 biến mất, mỗi cột ra 1 số.
Kết hợp điều kiện trong NumPy?	`(a > 1) & (b < 5)`, có ngoặc, không dùng `and`.
Quy tắc broadcasting?	So shape từ phải sang trái: bằng nhau hoặc một bên là 1.
Lát cắt `a[1:3]` là view hay copy?	View. Muốn độc lập thì `.copy()`.
Sửa lỗi "Expected 2D array"?	`a.reshape(-1, 1)`

---

13. 🚀 Đọc thêm

- NumPy: the absolute basics for beginners – hướng dẫn chính thức, có hình minh họa rất đẹp.
- Broadcasting – NumPy user guide – có hình động giải thích quy tắc.
- *Python Data Science Handbook*, chương 2 (Jake VanderPlas, đọc miễn phí tại jakevdp.github.io/PythonDataScienceHandbook).
- 100 NumPy exercises – 100 bài tập từ dễ đến khó, có đáp án.

Câu hỏi gợi mở cho bài sau: Bảng nhà đất thật có cột "Quận" (chữ), cột "Ngày đăng" (thời gian), có ô bị trống, và có hàng chục nghìn dòng. NumPy chỉ cho một kiểu dữ liệu trong mỗi mảng và gọi cột bằng số thứ tự `X[:, 2]` thay vì tên. Có công cụ nào cho phép gọi `df["quan"]`, trộn chữ với số, và hỏi "giá trung bình của **từng quận**" chỉ trong một dòng? Đó là **Pandas**, bài 1.4.

🔒 Đáp án trắc nghiệm

> Mẹo Notion: chọn tiêu đề này → `Turn into → Toggle heading 2` để ẩn đáp án.

Câu	Đáp án	Giải thích
1	**B**	Đúng như mục 4.1. A sai: phần lớn phép NumPy chạy trên 1 lõi. C, D sai: NumPy không nén, không bỏ số lẻ.
2	**C**	`(hàng, cột)` = (mẫu, đặc trưng). A đảo ngược. B sai: X có 2 chiều, 500 là `size`. D sai: `len(X)` = 100 (số hàng).
3	**B**	Có một số thực nên cả mảng nâng lên `float64`. A sai vì có `3.0`. C chỉ xảy ra với dữ liệu hỗn tạp kiểu đặc biệt. D sai: NumPy tự đổi kiểu, không báo lỗi.
4	**C**	Lát cắt là view. A (mặt nạ) và B (list chỉ số) luôn tạo bản sao. D hiển nhiên là bản sao.
5	**B**	`axis=0` gộp 2 hàng: `[1+4, 2+5, 3+6]`. A là `axis=1`. C là `sum()` không trục. D chỉ đúng khi có `keepdims=True`.
6	**C**	Dùng `&` và ngoặc. A, B báo lỗi "truth value ... ambiguous". D sai thứ tự ưu tiên: `18 & tuoi` được tính trước, gây lỗi.
7	**A**	`a[1:4]` = `[20 30 40]`, đảo ngược → `[40 30 20]`. B, D lấy sai biên. C quên đảo.
8	**C**	`v` shape `(3,)`; cần `(3, 1)` để kéo giãn theo hàng. A, B: `(3, 4)` với `(3,)` hay `(1, 3)` đều lỗi broadcasting. D: `.T` của mảng 1 chiều **không đổi gì**, vẫn `(3,)`.
9	**C**	`uint8` tràn số: 250 + 10 = 260 → 260 − 256 = 4. A sai vì uint8 không chứa được 260. B là kết quả nếu dùng `np.clip`. D sai: NumPy **không báo lỗi** khi tràn số trong phép toán mảng, đó là lý do nó nguy hiểm.
10	**B**	`S.mean(axis=1)` có shape `(3,)` = `[2, 5, 8]`, broadcast thành **hàng** `[2, 5, 8]` lặp lại, tức cột 0 trừ 2, cột 1 trừ 5, cột 2 trừ 8. Bảng vuông nên không lỗi (Lỗi 2, mục 7). A, D là bẫy. C vô căn cứ. Sửa: `S - S.mean(axis=1, keepdims=True)`.

🔒 Lời giải thực hành

> Mẹo Notion: chọn tiêu đề này → `Turn into → Toggle heading 2`.

Lời giải Bài 1

```
import numpy as np

ten = np.array(["An", "Bình", "Chi", "Dũng", "Em", "Giang"])
mon = np.array(["Toán", "Văn", "Anh"])
diem = np.array([
    [8.0, 6.5, 7.0],
    [5.5, 7.0, 6.0],
    [9.5, 8.0, 9.0],
    [4.0, 5.5, 3.5],
    [7.0, 9.0, 8.5],
    [6.0, 4.5, 7.5],
])
```

```
print(diem.shape)                                   # 1
print(diem.mean(axis=0).round(2))                   # 2: Toán, Văn, Anh
tb_hs = diem.mean(axis=1)
print(tb_hs.round(2))                               # 3
print("Giỏi nhất:", ten[tb_hs.argmax()])            # 4
print("Có môn dưới 5:", ten[(diem < 5).any(axis=1)])   # 5
print(dict(zip(mon.tolist(), ten[diem.argmax(axis=0)].tolist())))   # 6

diem_moi = diem.copy()                              # 7
diem_moi[:, 1] = np.clip(diem_moi[:, 1] + 0.5, 0, 10)
print(diem_moi[:, 1])

trong_so = np.array([2, 1, 1])                      # 8
print((diem @ trong_so / 4).round(2))
```

Kết quả mẫu:

```
(6, 3)
[6.67 6.75 6.92]
[7.17 6.17 8.83 4.33 8.17 6.  ]
Giỏi nhất: Chi
Có môn dưới 5: ['Dũng' 'Giang']
{'Toán': 'Chi', 'Văn': 'Em', 'Anh': 'Chi'}
[7.  7.5 8.5 6.  9.5 5. ]
[7.38 6.   9.   4.25 7.88 6.  ]
```

Lời giải Bài 2

```
import numpy as np

# Trên Colab: D = np.loadtxt("/content/sample_data/california_housing_train.csv",
delimiter=",", skiprows=1)
D = np.genfromtxt("https://download.mlcc.google.com/mledu-
datasets/california_housing_train.csv",
                  delimiter=",", skip_header=1)
print(D.shape)                                                  # 1

X, y = D[:, :8], D[:, 8]                                        # 2
np.set_printoptions(suppress=True)                              # in số thường,
không in dạng 1.2e+04
print(X.mean(axis=0).round(2))                                  # 3
print(X.min(axis=0).round(2))
print(X.max(axis=0).round(2))

print(y.max(), (y == y.max()).sum())                            # 4: giới hạn trần

phong_moi_ho = X[:, 3] / X[:, 6]                                # 5
print(np.median(phong_moi_ho).round(2))
```

```
print(y[X[:, 7] > 8].mean().round(0), y[X[:, 7] < 2].mean().round(0))   # 6

X_chuan = (X - X.mean(axis=0)) / X.std(axis=0)                  # 7
print(np.allclose(X_chuan.mean(axis=0), 0), np.allclose(X_chuan.std(axis=0), 1))

y_khong_tran = y[y < y.max()]                                   # 8: bỏ các khu bị
trần
print(y.mean().round(0), y_khong_tran.mean().round(0), (y.mean() -
y_khong_tran.mean()).round(0))
np.set_printoptions(suppress=False)                             # trả lại cài đặt
in mặc định
```

Kết quả mẫu:

```
(17000, 9)
[-119.56   35.63   28.59 2643.66  539.41 1429.57  501.22    3.88]
[-124.35   32.54    1.      2.      1.      3.      1.      0.5 ]
[ -114.31    41.95    52.   37937.    6445.   35682.    6082.      15.  ]
500001.0 814
5.24
460652.0 111357.0
True True
207301.0 192581.0 14720.0
```

Nhận xét:

- 814 khu có đúng giá 500.001 USD, chính là **giới hạn trần** (bạn sẽ gặp lại ở bài 1.4).
- Khu thu nhập cao có giá trung bình gấp **hơn 4 lần** khu thu nhập thấp: thu nhập là đặc trưng rất mạnh.
- Câu 8: bỏ 814 khu bị trần làm giá trung bình giảm gần 15.000 USD. Nhưng giá thật của các khu đó **còn cao hơn** 500.001, nên con số trung bình "đúng" thực ra còn cao hơn 207.301. Dữ liệu bị cắt trần làm mọi thống kê bị lệch, và không có cách sửa hoàn hảo. Ghi chú điều này lại khi làm báo cáo.

Lời giải Bài 3

```
import numpy as np
nha = np.array([
    [50, 2, 5], [65, 2, 3], [80, 3, 7], [100, 3, 2],
    [72, 3, 4], [90, 4, 6], [60, 1, 1], [78, 2, 4],
])
gia = np.array([2.5, 3.1, 4.0, 5.2, 3.6, 4.4, 3.0, 3.7])
mo_uoc = np.array([75, 3, 4])

# 1. Chuẩn hóa theo cột, nếu không thì diện tích (chênh hàng chục m²) "át" hết số
phòng (chênh 1–2)
tb, dlc = nha.mean(axis=0), nha.std(axis=0)
nha_c = (nha - tb) / dlc
mo_uoc_c = (mo_uoc - tb) / dlc                          # nhu cầu cũng phải chuẩn hóa
bằng CÙNG tb, dlc
```

```
kc = np.sqrt(((nha_c - mo_uoc_c) ** 2).sum(axis=1))   # khoảng cách Euclid tới
từng căn, shape (8,)
gan_nhat = np.argsort(kc)[:3]
print("3 căn giống nhất:", gan_nhat, nha[gan_nhat].tolist())
print("Khoảng cách:", kc[gan_nhat].round(2))
print("Giá đề xuất (TB 3 căn):", gia[gan_nhat].mean().round(2), "tỷ")

# 2. Ma trận khoảng cách: (8, 1, 3) - (1, 8, 3) → (8, 8, 3) rồi gộp trục cuối
hieu = nha_c[:, np.newaxis, :] - nha_c[np.newaxis, :, :]
M = np.sqrt((hieu ** 2).sum(axis=2))
print(M.shape, np.allclose(M, M.T), np.allclose(np.diag(M), 0))

M_tam = M + np.eye(8) * 1e9                          # "đẩy" đường chéo lên rất
lớn để không bị chọn
i, j = np.unravel_index(M_tam.argmin(), M_tam.shape) # đổi vị trí phẳng → (hàng,
cột)
print("Cặp giống nhất:", i, j, nha[i].tolist(), nha[j].tolist(), M[i, j].round(2))
```

Kết quả mẫu:

```
3 căn giống nhất: [4 7 1] [[72, 3, 4], [78, 2, 4], [65, 2, 3]]
Khoảng cách: [0.2  1.17 1.43]
Giá đề xuất (TB 3 căn): 3.47 tỷ
(8, 8) True True
Cặp giống nhất: 1 7 [65, 2, 3] [78, 2, 4] 1.01
```

Giải thích:

- Căn ở vị trí 4 (72 m^2, 3 phòng, 4 km) gần như khớp hoàn toàn nhu cầu. Sau đó là vị trí 7 (78 m^2, 2 phòng, 4 km) và vị trí 1 (65 m^2, 2 phòng, 3 km). Giá đề xuất là trung bình giá 3 căn này.
- Nếu **không chuẩn hóa**, top 3 thành vị trí 4, 7 và **2** (80 m^2, 3 phòng, **7 km**). Căn 2 xa trung tâm hơn nhu cầu tới 3 km, nhưng chênh 3 km "trông nhỏ" so với chênh hàng chục m^2, nên bị bỏ qua. Chuẩn hóa giúp mỗi đặc trưng có tiếng nói công bằng.
- `np.unravel_index` đổi vị trí trong mảng "duỗi phẳng" (do `argmin` trả về) thành cặp (hàng, cột).